Finite Volume Method - Finite Element Method Surface Processes Model

Reza Masoumzadeh

PhD Student in Computational Mathematics,
School of Mathematical Sciences,
Tongji University.

Email: RezaMasoumzadeh@tongji.edu.cn

November 2026

1 Surface Processes Model

Surface process models (SPMs) simulate the temporal evolution of topographic elevation, h, by applying material redistribution laws that govern erosion, sediment transport, and deposition. Different SPM formulations exist because the surface process considered to dominate landscape evolution varies among regions. In continental settings, fluvial processes and bedrock landsliding are generally regarded as the primary geomorphic controls, although glacial erosion and chemical weathering can also be significant in certain environments.

The evolution of the surface h is defined via the following advection-diffusion equation:

$$\begin{aligned} &\frac{\partial h}{\partial t} + \boldsymbol{u}_H \cdot \nabla_H h = u_V + \nabla_H \cdot (\kappa(\mathbf{x}) \nabla_H h), \quad \text{in } \Omega, \\ &\frac{\partial h}{\partial \boldsymbol{n}} = 0, \qquad\qquad\qquad\qquad\qquad\qquad \text{on } \partial\Omega, \end{aligned} \tag{1}$$

where $\boldsymbol{u}$ is the velocity, sub-script H denotes quantities (or operators) defined in the horizontal plane, u_V denotes the component of the velocity in the vertical direction, $\kappa(\mathbf{x})$ represents a diffusivity, and $\boldsymbol{n}$ is the outward-pointing unit normal vector.

Surface erosion occurs in regions of negative topographic curvature ($\nabla_H^2 h < 0$), where the surface elevation decreases due to diffusive smoothing. Conversely, sedimentation occurs in regions of positive topographic curvature ($\nabla_H^2 h > 0$), where the surface elevation increases as material is redistributed into topographic depressions.

To solve Eq. (1), we adopt a first-order operator splitting technique in which the advective and diffusive and source terms are updated sequentially. The operator splitting technique updates the height h in the following manner: Solve the advection equation

$$\frac{\partial \widehat{h}}{\partial t} + \boldsymbol{u}_H \cdot \nabla_H \widehat{h} = 0, \qquad t \in [t^k, t^k + \Delta t] \tag{2}$$

for the advected height $\widehat{h}$ using the following initial data $\widehat{h}(\mathbf{x}, t^k) = h(\mathbf{x}, t^k)$. Then solve the diffusion equation

$$\frac{\partial h}{\partial t} = u_V(\mathbf{x}, t) + \nabla_H \cdot (\kappa(h, \mathbf{x}, t) \nabla_H h), \qquad t \in [t^k, t^k + \Delta t] \tag{3}$$

for $h(\mathbf{x}, t)$ using initial data $h(\mathbf{x}, t^k) = \widehat{h}(\mathbf{x}, t^k + \Delta t)$.

Forward Euler

We adopt an explicit (in time) finite difference approximation to solve Eq. (3), which given some initial height $h^k(\mathbf{x})$ defined at time $t = t^k$ yields

$$\frac{h^{k+1} - h^k}{\Delta t} = u_V(\mathbf{x}, t^k) + \nabla_H \cdot (\kappa(h^k, \mathbf{x}, t^k)\nabla_H h^k), \tag{4}$$

where Δt is the time-step and h^{k+1} is the approximate value of $h(t^k + \Delta t)$. Note the non-linearity is evaluated using values from the previous time step k. The hyperbolic equation defined by Eq. (2) is solved using a semi-Lagrangian technique.

2 Finite Element Method for Solving the Diffusion Equation

We consider the following time-discrete surface evolution equation:

$$\frac{h^{k+1} - h^k}{\Delta t} = u_V(\mathbf{x}, t^k) + \nabla_H \cdot \left(\kappa(h^k, \mathbf{x}, t^k)\nabla_H h^k\right). \tag{5}$$

Here, h denotes the surface elevation, u_V is the vertical velocity, κ is the surface diffusivity, and ∇_H denotes the horizontal gradient operator.

2.1 Constant Diffusivity κ

For the first FESPM implementation, we consider a constant diffusivity,

$$\kappa(h^k, \mathbf{x}, t^k) = \kappa, \tag{6}$$

so that Eq. (5) becomes

$$\frac{h^{k+1} - h^k}{\Delta t} = u_V^k + \kappa \nabla_H^2 h^k. \tag{7}$$

Multiplying Eq. (7) by a test function v, and integrating over the computational domain Ω, gives

$$\int_\Omega \frac{h^{k+1} - h^k}{\Delta t} v \, d\Omega = \int_\Omega u_V^k v \, d\Omega + \int_\Omega \kappa \nabla_H^2 h^k v \, d\Omega. \tag{8}$$

Using integration by parts for the diffusion term,

$$\int_\Omega \kappa \nabla_H^2 h^k v \, d\Omega = -\int_\Omega \kappa \nabla_H h^k \cdot \nabla_H v \, d\Omega + \int_\Gamma (\kappa \nabla_H h^k \cdot \mathbf{n}) v \, d\Gamma. \tag{9}$$

For homogeneous Neumann boundary conditions,

$$\frac{\partial h}{\partial n} = 0 \qquad \text{on } \Gamma, \tag{10}$$

the boundary integral vanishes,

$$\int_{\Gamma} (\kappa \nabla_H h^k \cdot \mathbf{n}) v \, d\Gamma = 0. \tag{11}$$

Therefore, the weak formulation becomes

$$\int_{\Omega} \frac{h^{k+1} - h^k}{\Delta t} v \, d\Omega + \int_{\Omega} \kappa \nabla_H h^k \cdot \nabla_H v \, d\Omega = \int_{\Omega} u_V^k v \, d\Omega. \tag{12}$$

• If we consider a Neumann boundary condition as:

$$\kappa \frac{\partial h}{\partial n} = q_N \qquad \text{on } \Gamma_N, \tag{13}$$

therefore, the weak form is

$$\boxed{\int_{\Omega} \frac{h^{k+1} - h^k}{\Delta t} v \, d\Omega + \int_{\Omega} \kappa \nabla_H h^k \cdot \nabla_H v \, d\Omega = \int_{\Omega} u_V^k v \, d\Omega + \int_{\Gamma} q_N \, v \, d\Gamma} \tag{14}$$

Finite Element Approximation

The surface elevation is approximated using finite element basis functions,

$$h^k(\mathbf{x}) \approx \sum_{j=1}^{N} H_j^k \phi_j(\mathbf{x}), \tag{15}$$

and similarly,

$$h^{k+1}(\mathbf{x}) \approx \sum_{j=1}^{N} H_j^{k+1} \phi_j(\mathbf{x}), \tag{16}$$

where N is the total number of FEM nodes, ϕ_j are the finite element basis functions, and H_j^k represents the numerical value of h at node j at time level k.

Using the Galerkin method, the test function is chosen as

$$v = \phi_i. \tag{17}$$

Substituting Eqs. (73)–(17) into the weak formulation gives

$$\int_\Omega \frac{\sum_{j=1}^{N} H_j^{k+1}\phi_j - \sum_{j=1}^{N} H_j^k \phi_j}{\Delta t}\phi_i \, d\Omega + \int_\Omega \kappa \left(\sum_{j=1}^{N} H_j^k \nabla_H \phi_j \right) \cdot \nabla_H \phi_i \, d\Omega = \int_\Omega u_V^k \phi_i \, d\Omega. \quad (18)$$

Because the summations and integrals are linear,

$$\frac{1}{\Delta t}\sum_{j=1}^{N} \left(H_j^{k+1} - H_j^k\right) \int_\Omega \phi_j \phi_i \, d\Omega + \sum_{j=1}^{N} H_j^k \int_\Omega \kappa \nabla_H \phi_j \cdot \nabla_H \phi_i \, d\Omega = \int_\Omega u_V^k \phi_i \, d\Omega. \quad (19)$$

Mass Matrix

The FEM mass matrix is defined as

$$M_{ij} = \int_\Omega \phi_i \phi_j \, d\Omega. \quad (20)$$

Therefore,

$$\frac{1}{\Delta t}\sum_{j=1}^{N} M_{ij}\left(H_j^{k+1} - H_j^k\right) = \frac{1}{\Delta t}\left[M\left(\boldsymbol{H}^{k+1} - \boldsymbol{H}^k\right)\right]_i. \quad (21)$$

Diffusion Stiffness Matrix

The diffusion stiffness matrix is defined as

$$K_{ij} = \int_\Omega \kappa \nabla_H \phi_j \cdot \nabla_H \phi_i \, d\Omega. \quad (22)$$

Consequently,

$$\sum_{j=1}^{N} K_{ij} H_j^k = \left[K\boldsymbol{H}^k\right]_i. \quad (23)$$

Vertical Velocity Load Vector

The vertical velocity produces the load vector

$$F_i^k = \int_\Omega u_V^k \phi_i \, d\Omega. \tag{24}$$

Therefore,

$$\mathbf{F}^k = \left[F_1^k, F_2^k, \ldots, F_N^k\right]^T. \tag{25}$$

Derivation of $\mathbf{F}^k = \mathbf{M}\mathbf{u}_V^k$

When the source term u_V^k is defined by discrete nodal values rather than a continuous analytical function, it is interpolated using the same shape functions $\phi_j(\mathbf{x})$:

$$u_V^k(\mathbf{x}) \approx \sum_{j=1}^{N} (u_V^k)_j \phi_j(\mathbf{x}) \tag{26}$$

Substituting this interpolation into the definition of the load vector component F_i^k:

$$\begin{aligned} F_i^k &= \int_\Omega \phi_i(\mathbf{x}) u_V^k(\mathbf{x}) \, d\Omega & (27)\\ &= \int_\Omega \phi_i(\mathbf{x}) \left(\sum_{j=1}^{N} (u_V^k)_j \phi_j(\mathbf{x}) \right) d\Omega & (28)\\ &= \sum_{j=1}^{N} \left(\int_\Omega \phi_i(\mathbf{x}) \phi_j(\mathbf{x}) \, d\Omega \right) (u_V^k)_j & (29) \end{aligned}$$

Since the mass matrix is defined as $M_{ij} = \int_\Omega \phi_i(\mathbf{x}) \phi_j(\mathbf{x}) \, d\Omega$, the expression simplifies to:

$$F_i^k = \sum_{j=1}^{N} M_{ij} (u_V^k)_j \implies \mathbf{F}^k = \mathbf{M}\mathbf{u}_V^k \tag{30}$$

Benefit: This avoids recalculating spatial integrals at each time step, replacing numerical quadrature with a fast matrix-vector multiplication ($\mathbf{M}\mathbf{u}_V^k$).

Final FEM System

Combining the mass matrix, stiffness matrix, and vertical velocity load vector gives

$$\frac{1}{\Delta t}\mathbf{M}\left(\boldsymbol{H}^{k+1}-\boldsymbol{H}^{k}\right)+\mathbf{K}\boldsymbol{H}^{k}=\mathbf{F}^{k}. \tag{31}$$

Multiplying by Δt,

$$\mathbf{M}\left(\boldsymbol{H}^{k+1}-\boldsymbol{H}^{k}\right)=\Delta t\left(\mathbf{F}^{k}-\mathbf{K}\boldsymbol{H}^{k}\right). \tag{32}$$

Thus,

$$\mathbf{M}\boldsymbol{H}^{k+1}=\mathbf{M}\boldsymbol{H}^{k}+\Delta t\left(\mathbf{F}^{k}-\mathbf{K}\boldsymbol{H}^{k}\right). \tag{33}$$

Iterative Solution of the Linear Systems

Overview of Iterative Methods

Discretization of the surface diffusion equation by the finite element method (FEM) leads, at every time step, to a large sparse linear system of the form

$$A\mathbf{x} = \mathbf{b}, \tag{34}$$

where $A \in \mathbb{R}^{n\times n}$ is a sparse, symmetric, and positive definite matrix, $\mathbf{b} \in \mathbb{R}^n$ is the right-hand side, and n denotes the number of mesh nodes. For dense linear systems, direct methods such as Gaussian elimination and Cholesky factorization generally require $\mathcal{O}(n^3)$ computational operations and $\mathcal{O}(n^2)$ storage. Although sparse direct solvers exploit the sparsity of finite element matrices, their computational cost can still become substantial due to fill-in during factorization, particularly for large-scale problems. Iterative methods, in contrast, exploit the sparsity of A, primarily require matrix–vector products, and can be computationally efficient for large-scale systems.

Classical stationary iterative methods—Jacobi, Gauss–Seidel, and successive over-relaxation (SOR)—are derived by splitting the matrix $A = M - N$ and iterating

$$M\mathbf{x}^{(k+1)} = N\mathbf{x}^{(k)} + \mathbf{b}, \tag{35}$$

where M is chosen to be easily invertible. Although simple to implement, these methods generally exhibit linear convergence, and their convergence rates may deteriorate as the mesh is refined, particularly for elliptic problems.

Krylov subspace methods constitute a more powerful class of iterative solvers. They construct the k-th iterate $\mathbf{x}^{(k)}$ within the affine Krylov subspace

$$\mathcal{K}_k(A, \mathbf{r}_0) = \operatorname{span}\left\{\mathbf{r}_0, A\mathbf{r}_0, A^2\mathbf{r}_0, \ldots, A^{k-1}\mathbf{r}_0\right\}, \tag{36}$$

where $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}^{(0)}$ is the initial residual. Within this subspace, the approximate solution is determined by an appropriate optimality condition, such as minimizing the error or residual in a suitable norm. Unlike methods that explicitly construct and orthogonalize a complete basis of the Krylov subspace, certain Krylov methods exploit short recurrence relations to reduce computational and memory requirements. In particular, the Conjugate Gradient method takes advantage of the symmetry and positive definiteness of A, making it especially suitable for symmetric positive definite linear systems.

The Conjugate Gradient Method

Problem Formulation

Let $A \in \mathbb{R}^{n \times n}$ be symmetric and positive definite (SPD), i.e., $A = A^\top$ and

$$\mathbf{v}^\top A \mathbf{v} > 0 \qquad \text{for all } \mathbf{v} \neq \mathbf{0}. \tag{37}$$

Solving $A\mathbf{x} = \mathbf{b}$ is then equivalent to minimizing the quadratic functional

$$\Phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top A \mathbf{x} - \mathbf{x}^\top \mathbf{b}, \qquad \mathbf{x} \in \mathbb{R}^n, \tag{38}$$

whose unique minimizer satisfies

$$\nabla \Phi(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = \mathbf{0}, \tag{39}$$

i.e., the solution of the linear system. The level sets of Φ are ellipsoids centered at $\mathbf{x}^\star = A^{-1}\mathbf{b}$, whose eccentricity is governed by the spectral condition number of A.

Variational and Geometric Interpretation

The Conjugate Gradient method minimizes Φ along a sequence of mutually A-conjugate (or A-orthogonal) search directions $\{\mathbf{p}_0, \mathbf{p}_1, \ldots\}$, which satisfy

$$\mathbf{p}_i^\top A \mathbf{p}_j = 0, \qquad i \neq j. \tag{40}$$

A-conjugacy generalizes the notion of orthogonality with respect to the inner product induced by A, namely

$$\langle \mathbf{u}, \mathbf{v} \rangle_A = \mathbf{u}^\top A \mathbf{v}. \tag{41}$$

Within this inner product, the error $\mathbf{e}^{(k)} = \mathbf{x}^\star - \mathbf{x}^{(k)}$ is minimized over the affine Krylov subspace

$$\mathbf{x}^{(0)} + \mathcal{K}_k(A, \mathbf{r}_0). \tag{42}$$

Thus, the CG method produces the best approximation in the A-norm among all vectors in this affine subspace, where

$$\|\mathbf{e}\|_A = \sqrt{\mathbf{e}^\top A \mathbf{e}}. \tag{43}$$

This variational property is the defining characteristic of CG and guarantees a non-increasing A-norm of the error in exact arithmetic.

Algorithm

Starting from an initial guess $\mathbf{x}^{(0)}$ (typically $\mathbf{x}^{(0)} = \mathbf{0}$), the initial residual and search direction are defined as

$$\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)}, \qquad \mathbf{p}^{(0)} = \mathbf{r}^{(0)}. \tag{44}$$

At iteration k, the step length α_k, the solution and residual updates, and the new search direction are given by

$$\alpha_k = \frac{\mathbf{r}^{(k)\top}\mathbf{r}^{(k)}}{\mathbf{p}^{(k)\top}A\mathbf{p}^{(k)}}, \tag{45}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}, \tag{46}$$

$$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k A\mathbf{p}^{(k)}, \tag{47}$$

$$\beta_k = \frac{\mathbf{r}^{(k+1)\top}\mathbf{r}^{(k+1)}}{\mathbf{r}^{(k)\top}\mathbf{r}^{(k)}}, \tag{48}$$

$$\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}. \tag{49}$$

The iterations are terminated when the prescribed convergence criterion is satisfied. Each iteration requires one matrix–vector product $A\mathbf{p}^{(k)}$, two inner products, and several vector updates. Consequently, the computational cost per iteration is primarily determined by the number of nonzero entries in A. In exact arithmetic, CG terminates in at most n iterations. In floating-point arithmetic, convergence is typically monitored using the relative residual criterion

$$\left\|\mathbf{r}^{(k)}\right\|_2 \leq \varepsilon \left\|\mathbf{r}^{(0)}\right\|_2, \tag{50}$$

where ε is a prescribed tolerance. For well-conditioned or preconditioned systems, the number of iterations required to achieve convergence can be substantially smaller than n. The short recurrence relations are a distinguishing feature of CG and account for its relatively low memory requirements compared with Krylov methods that require full orthogonalization.

Convergence Properties

A classical convergence bound for CG in the A-norm of the error is

$$\left\|\mathbf{e}^{(k)}\right\|_A \leq 2\left(\frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1}\right)^k \left\|\mathbf{e}^{(0)}\right\|_A, \tag{51}$$

where

$$\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)} \tag{52}$$

is the spectral condition number of A. This bound indicates that CG converges more rapidly for well-conditioned matrices and that convergence may deteriorate as the condition number increases.

For consistent mass matrices arising from P_1 finite elements on shape-regular, quasi-uniform meshes, the spectral condition number is generally bounded independently of the mesh size. Consequently, the mass matrix is typically well-conditioned, allowing CG to converge in a relatively small number of iterations without preconditioning. The precise conditioning, however, depends on the mesh geometry and regularity.

When preconditioning is employed, the resulting method is referred to as the Preconditioned Conjugate Gradient (PCG) method. For an SPD preconditioner P, the preconditioned residual is defined as

$$\mathbf{z}^{(k)} = P^{-1}\mathbf{r}^{(k)}. \tag{53}$$

The corresponding recurrence coefficients become

$$\alpha_k = \frac{\mathbf{r}^{(k)\top}\mathbf{z}^{(k)}}{\mathbf{p}^{(k)\top}A\mathbf{p}^{(k)}}, \tag{54}$$

$$\beta_k = \frac{\mathbf{r}^{(k+1)\top}\mathbf{z}^{(k+1)}}{\mathbf{r}^{(k)\top}\mathbf{z}^{(k)}}, \tag{55}$$

with the search direction updated according to

$$\mathbf{p}^{(k+1)} = \mathbf{z}^{(k+1)} + \beta_k\mathbf{p}^{(k)}. \tag{56}$$

Preconditioning aims to improve the spectral properties of the system and accelerate convergence while preserving the symmetry and positive definiteness required by CG.

Application to the Surface Diffusion Problem

In the present finite element formulation, the diffusion step is advanced explicitly with respect to the stiffness operator. Accordingly, the discrete problem at each time level is written as

$$\mathbf{M}\boldsymbol{H}^{k+1} = \mathbf{M}\boldsymbol{H}^{k} + \Delta t \left(\mathbf{F}^{k} - \mathbf{K}^{k}\boldsymbol{H}^{k}\right), \tag{57}$$

where $\mathbf{M}$ and $\mathbf{K}^{k}$ denote the consistent mass and stiffness matrices, respectively, $\boldsymbol{H}^{k}$ is the surface elevation at time level k, $\mathbf{F}^{k}$ is the forcing vector at time level k, including contributions from vertical velocity and other source terms, and Δt is the time step. The stiffness matrix may depend on the current surface elevation through the nonlinear diffusivity law.

The above equation can be expressed in the standard linear system form

$$A\mathbf{x} = \mathbf{b}, \tag{58}$$

with

$$A = \mathbf{M}, \qquad \mathbf{x} = \boldsymbol{H}^{k+1}, \tag{59}$$

and

$$\mathbf{b} = \mathbf{M}\boldsymbol{H}^{k} + \Delta t \left(\mathbf{F}^{k} - \mathbf{K}^{k}\boldsymbol{H}^{k}\right). \tag{60}$$

Because the consistent mass matrix $\mathbf{M}$ is symmetric positive definite for the standard P_1 finite element formulation, the Conjugate Gradient method is directly applicable to this linear system. The matrix–vector products involving $\mathbf{M}$ and $\mathbf{K}^{k}$ are evaluated using a fixed sparse-row storage format. The number of nonzero entries per row depends on the connectivity of the underlying triangular mesh and its node ordering. For the structured mesh employed in the present implementation, each row contains at most nine nonzero entries.

Thus, the linear system is solved entirely by CG applied to the mass matrix, whereas the stiffness operator contributes only to the right-hand-side evaluation. For the nonlinear diffusivity laws considered in this work, the stiffness matrix is reassembled at every time step using the current surface elevation, while the mass matrix is assembled once during initialization, provided that the computational mesh remains fixed.

The largest generalized eigenvalue of the matrix pair $(\mathbf{K}^{k}, \mathbf{M})$ is also estimated through power iteration to determine the stability limit of the explicit diffusion scheme. At each power iteration, the resulting linear system involving the mass matrix is solved using CG. This procedure is appropriate because the mass matrix remains symmetric positive defi-

nite, even when the stiffness matrix is positive semidefinite under homogeneous Neumann boundary conditions.

In all cases, the CG iteration is initialized with $\mathbf{x}^{(0)} = \mathbf{0}$ and terminated when the relative residual satisfies

$$\frac{\|\mathbf{r}^{(k)}\|_2}{\|\mathbf{r}^{(0)}\|_2} \leq \varepsilon, \qquad \varepsilon = 10^{-10}, \tag{61}$$

provided that the initial residual is nonzero. This tolerance is selected to ensure that the algebraic solution error remains sufficiently small compared with the discretization error of the overall numerical scheme.

In the present formulation, the linear system is governed by the consistent finite element mass matrix $\mathbf{M}$. For P_1 elements, this matrix is symmetric positive definite and well-conditioned, with a spectral condition number $\kappa(\mathbf{M}) = \mathcal{O}(1)$ that is independent of mesh resolution. As a result, the standard Conjugate Gradient (CG) method converges rapidly within a few iterations. Employing a Preconditioned Conjugate Gradient (PCG) scheme would introduce unnecessary computational and memory overhead without significantly improving convergence rates. While preconditioning is often essential for the ill-conditioned stiffness matrix, it is redundant here. Thus, the plain CG method is the most computationally efficient choice for this system.

Stability Analysis

1. Explicit Scheme

The explicit time integration yields the update formulation:

$$\boldsymbol{H}^{k+1} = \left(\mathbf{I} - \Delta t \mathbf{M}^{-1}\mathbf{K}\right) \boldsymbol{H}^{k} + \Delta t \mathbf{M}^{-1}\mathbf{F}^{k} \tag{62}$$

Linear stability requires the spectral radius of the amplification matrix $\mathbf{G}_{\text{exp}} = \mathbf{I} - \Delta t \mathbf{M}^{-1}\mathbf{K}$ to be bounded by unity ($|\lambda(\mathbf{G}_{\text{exp}})| \leq 1$):

$$|1 - \Delta t \lambda_i(\mathbf{M}^{-1}\mathbf{K})| \leq 1 \implies \Delta t \leq \frac{2}{\lambda_{\max}(\mathbf{M}^{-1}\mathbf{K})} \tag{63}$$

Note: The explicit method is *conditionally stable*. Because $\lambda_{\max} \propto \Delta x^{-2}$, spatial mesh refinement severely restricts the permissible time step ($\Delta t \propto \Delta x^2$).

2.2 Non-Linear Variable Diffusivity κ

The diffusivity κ is non-linear and spatially variable as it depends on the local surface slope magnitude $\|\nabla_H h\|$:

$$\kappa(\mathbf{x}) = \kappa_0 \left[1 - \left(\frac{\|\nabla_H h\|}{S_c}\right)^{n_s}\right]^{-1} \tag{64}$$

Parameters:

- κ_0: Base constant diffusivity coefficient.
- S_c: Critical threshold slope ($S_c = \tan\theta$).
- n_s: Non-linear exponent (commonly $n_s = 2$).
- $\|\nabla_H h\| = \sqrt{\left(\frac{\partial h}{\partial x}\right)^2 + \left(\frac{\partial h}{\partial z}\right)^2}$: Gradient magnitude.

FEM Impact: Because $\kappa = \kappa(h)$, the stiffness matrix $\mathbf{K}(h)$ depends on the state variable h, making the PDE non-linear. Consequently, κ must be re-evaluated at each Gauss quadrature point during numerical integration.

Weak Form Derivation for Nonlinear Diffusion

1. Strong Form

The governing diffusion equation is:

$$\frac{\partial h}{\partial t} = u_V + \nabla_H \cdot (\kappa \nabla_H h) \tag{65}$$

where ∇_H is the horizontal gradient operator.

2. Weighted Residual Form

We multiply by a test function v and integrate over the domain Ω:

$$\int_\Omega v \frac{\partial h}{\partial t}\, d\Omega = \int_\Omega v u_V\, d\Omega + \int_\Omega v \Big[\nabla_H \cdot (\kappa \nabla_H h)\Big]\, d\Omega \tag{66}$$

3. Integration by Parts & Divergence Theorem

Using the product rule, the diffusion term is expanded:

$$\int_\Omega v\Big[\nabla_H \cdot (\kappa \nabla_H h)\Big]\, d\Omega = \int_\Omega \nabla_H \cdot (v\kappa \nabla_H h)\, d\Omega - \int_\Omega \kappa \nabla_H v \cdot \nabla_H h\, d\Omega \tag{67}$$

Applying the Divergence Theorem to the first term on the right-hand side:

$$\int_\Omega \nabla_H \cdot (v\kappa \nabla_H h)\, d\Omega = \oint_{\partial\Omega} v\kappa \nabla_H h \cdot \mathbf{n}\, dS \tag{68}$$

Assuming no-flux boundary conditions (the boundary integral vanishes), we obtain the weak form:

$$\int_\Omega v \frac{\partial h}{\partial t}\, d\Omega + \int_\Omega \kappa \nabla_H v \cdot \nabla_H h\, d\Omega = \int_\Omega v u_V\, d\Omega \tag{69}$$

4. Constant vs. Non-Constant κ

If κ is a constant, it can be factored out of the integral:

$$\int_\Omega \kappa \nabla_H v \cdot \nabla_H h\, d\Omega = \kappa \int_\Omega \nabla_H v \cdot \nabla_H h\, d\Omega \tag{70}$$

However, if κ is a function, it **must** remain inside the integral because it varies spatially over the domain Ω.

In this model, κ is highly nonlinear and depends on the local gradient of h:

$$\kappa(\mathbf{x}) = \kappa_0 \left[1 - \left(\frac{\|\nabla_H h\|}{S_c}\right)^{n_s}\right]^{-1} \tag{71}$$

where the gradient magnitude is:

$$\|\nabla_H h\| = \sqrt{\left(\frac{\partial h}{\partial x}\right)^2 + \left(\frac{\partial h}{\partial z}\right)^2} \tag{72}$$

Because κ depends on $\nabla_H h$, it must be evaluated at every Gauss quadrature point during the numerical integration of the element matrices.

5. FEM approximation on P1 triangles

On the structured grid, each quadrilateral cell is split into two P_1 triangles. On each triangle T_e with vertices $\{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}$ the field h is approximated by

$$h^k(\mathbf{x}) \;\approx\; \sum_{m=1}^{N} H_m^k \, \phi_m(\mathbf{x}), \tag{73}$$

where ϕ_m are the global linear (barycentric) shape functions, H_m^k are the global nodal values, and N is the total number of nodes. The summation index is called m to distinguish it from the local indices $i, j \in \{1, 2, 3\}$ used below.

When restricted to a single triangle T_e, only three of the shape functions in (73) are non-zero. Relabeling them by a local index $i \in \{1, 2, 3\}$:

$$h^k(\mathbf{x})\big|_{T_e} \;=\; H_1^k \, \phi_1 + H_2^k \, \phi_2 + H_3^k \, \phi_3, \tag{74}$$

where ϕ_i denotes the linear shape function associated with the i-th local vertex of T_e. Because the ϕ_i are linear, their gradients — and therefore $\nabla_H h$ — are constant on each element.

6. Gradient of the shape functions

Let A_e be the (signed) area of the triangle and let (i, m, n) be a cyclic permutation of $(1, 2, 3)$. Define the geometric coefficients

$$b_i = z_m - z_n, \qquad c_i = x_n - x_m, \tag{75}$$

so that, for $i = 1, 2, 3$,

$$\begin{aligned} b_1 &= z_2 - z_3, & c_1 &= x_3 - x_2, \\ b_2 &= z_3 - z_1, & c_2 &= x_1 - x_3, \\ b_3 &= z_1 - z_2, & c_3 &= x_2 - x_1. \end{aligned} \tag{76}$$

The linear shape function associated with vertex i is

$$\phi_i(x,z) = \frac{1}{2A_e}\Big[\, b_i\,(x - x_m) + c_i\,(z - z_m)\,\Big]. \tag{77}$$

This form satisfies, by construction,

$$\phi_i(x_m, z_m) = \phi_i(x_n, z_n) = 0, \qquad \phi_i(x_i, z_i) = 1. \tag{78}$$

Equivalently, (77) can be written as $\phi_i(x,z) = \big(a_i + b_i x + c_i z\big)/(2A_e)$, with $a_i = x_m z_n - x_n z_m$; the constant $a_i/(2A_e)$ fixes the normalisation $\phi_i(x_i, z_i) = 1$.

Differentiating (77),

$$\nabla_H \phi_i \;=\; \begin{pmatrix} \partial_x \phi_i \\ \partial_z \phi_i \end{pmatrix} \;=\; \frac{1}{2A_e} \begin{pmatrix} b_i \\ c_i \end{pmatrix}, \qquad i = 1,2,3, \tag{79}$$

which is a *constant* vector on T_e.

7. Gradient of h on a triangle

Substituting (74) into $\nabla_H h$ and using (79):

$$\nabla_H h\big|_{T_e} \;=\; \sum_{i=1}^{3} H_i^k\, \nabla_H \phi_i \;=\; \frac{1}{2A_e} \sum_{i=1}^{3} H_i^k \begin{pmatrix} b_i \\ c_i \end{pmatrix}. \tag{80}$$

Equivalently, in components:

$$\left(\frac{\partial h}{\partial x}\right)_e = \frac{1}{2A_e}\big(b_1 H_1^k + b_2 H_2^k + b_3 H_3^k\big), \tag{81}$$

$$\left(\frac{\partial h}{\partial z}\right)_e = \frac{1}{2A_e}\big(c_1 H_1^k + c_2 H_2^k + c_3 H_3^k\big). \tag{82}$$

Because b_i and c_i depend only on the triangle geometry, the gradient (80) is evaluated **once per triangle** from the three nodal values H_1^k, H_2^k, H_3^k — no quadrature is required.

8. Magnitude of the gradient

The scalar quantity that appears in the critical-slope law is

$$\|\nabla_H h\|_e \;=\; \sqrt{\left(\frac{\partial h}{\partial x}\right)_e^2 + \left(\frac{\partial h}{\partial z}\right)_e^2}. \tag{83}$$

Using (81)–(82):

$$\|\nabla_H h\|_e \;=\; \frac{1}{2A_e}\sqrt{\Big(\sum_{i=1}^{3} b_i H_i^k\Big)^2 + \Big(\sum_{i=1}^{3} c_i H_i^k\Big)^2}. \tag{84}$$

9. Element diffusivity κ_e

Since $\|\nabla_H h\|_e$ is constant on T_e, the diffusivity κ is also constant on that element; we denote it by κ_e. Using the critical-slope law

$$\kappa(\mathbf{x}, h) = \kappa_0 \left[1 - \left(\frac{\|\nabla_H h\|}{S_c}\right)^{n_s}\right]^{-1}, \tag{85}$$

we obtain the *element scalar*

$$\boxed{\kappa_e \;=\; \kappa_0 \left[1 - \left(\frac{\|\nabla_H h\|_e}{S_c}\right)^{n_s}\right]^{-1}} \tag{86}$$

with $\|\nabla_H h\|_e$ given by (84). In practice the ratio $\|\nabla_H h\|_e / S_c$ is capped below 1 (e.g. at $1 - 10^{-5}$) to avoid the singularity as the slope approaches the critical value.

When κ_0, S_c or n_s vary spatially, their values on element T_e are taken as the arithmetic mean over the three vertices:

$$\kappa_0 \;\to\; \tfrac{1}{3}\sum_{i=1}^{3} \kappa_{0,i}, \qquad S_c \;\to\; \tfrac{1}{3}\sum_{i=1}^{3} S_{c,i}, \qquad n_s \;\to\; \tfrac{1}{3}\sum_{i=1}^{3} n_{s,i}. \tag{87}$$

10. Local stiffness matrix

The local stiffness matrix on T_e is

$$K_{ij}^{(e)} = \int_{T_e} \kappa \, \nabla_H \phi_i \cdot \nabla_H \phi_j \, d\Omega, \qquad i, j \in \{1, 2, 3\}. \tag{88}$$

Since both $\kappa = \kappa_e$ and $\nabla_H \phi_i \cdot \nabla_H \phi_j$ are constant on T_e, the integral reduces to a single multiplication by the area:

$$\boxed{K_{ij}^{(e)} = \frac{\kappa_e}{4\,A_e}\big(b_i b_j + c_i c_j\big)} \qquad i, j = 1, 2, 3. \tag{89}$$

This is the exact value of the integral for P_1 triangles — no numerical quadrature is required.

11. Assembly and time dependence

The global stiffness matrix is assembled by summing the local matrices over all triangles sharing each pair of nodes,

$$K_{MN} = \sum_{e} \sum_{\substack{i,j: \\ \mathrm{glob}(i)=M,\, \mathrm{glob}(j)=N}} K_{ij}^{(e)}, \tag{90}$$

where M, N denote global node numbers and $\mathrm{glob}(\cdot)$ maps the local index to the global index.

Because κ_e in (86) depends on $\|\nabla_H h\|_e$, which in turn depends on the nodal values H_i^k, the matrix K is *nonlinear* in h. The temporal discretisation therefore treats the diffusion operator explicitly:

$$\boldsymbol{M}\,\boldsymbol{H}^{n+1} = \boldsymbol{M}\,\boldsymbol{H}^{*} + \Delta t \left(\boldsymbol{M}\,\boldsymbol{U}_V - \boldsymbol{K}\big(\boldsymbol{H}^{*}\big)\,\boldsymbol{H}^{*}\right), \tag{91}$$

where $\boldsymbol{H}^{*}$ is the post-advection surface. The sequence at each diffusion step is:

1. For every triangle T_e, compute b_i and c_i from (75).

2. Compute $\nabla_H h|_{T_e}$ from (81)–(82).

3. Compute $\|\nabla_H h\|_e$ from (84).

4. Compute κ_e from (86).

5. Build the local matrix $K_{ij}^{(e)}$ from (89).

6. Assemble $K_{ij}^{(e)}$ into the global matrix K_{MN}.

7. Solve the linear system (91) with the consistent mass matrix $\boldsymbol{M}$.

Steps 2–6 are repeated at **every** diffusion step, because h, and therefore κ_e, changes with time. The cost is $O(N_{\text{elem}})$, negligible compared with the CG solve of (91).

3 Finite Volume Method for Solving the Advection Equation

3.1 Basic Principles of the Finite Volume Method

The Finite Volume Method (FVM) is a numerical method for solving partial differential equations by enforcing integral balance laws over a set of non-overlapping control volumes. Instead of approximating the governing equation directly at individual points, FVM integrates the equation over each control volume and evaluates the transport of the physical quantity through its boundary faces. This approach is particularly suitable for transport problems, such as advection and diffusion, because it provides a conservative discretization of the governing equations.

Let Ω denote the computational domain, which is partitioned into N non-overlapping control volumes Ω_i:

$$\Omega = \bigcup_{i=1}^{N} \Omega_i, \qquad \Omega_i \cap \Omega_j = \varnothing \quad \text{for } i \neq j, \tag{92}$$

apart from shared boundaries. The area of control volume Ω_i is denoted by $|\Omega_i|$, and its boundary is denoted by $\partial\Omega_i$.

The fundamental idea of FVM is to integrate the governing equation over each control volume. The divergence theorem is then used to transform volume integrals of flux divergences into boundary integrals. These boundary integrals are approximated by numerical fluxes across the control-volume faces. Since neighboring cells share the same face flux with opposite orientations, the method preserves the discrete conservation property.

For a general conservation equation,

$$\frac{\partial q}{\partial t} + \nabla \cdot \boldsymbol{F} = 0, \tag{93}$$

where q is the transported quantity and $\boldsymbol{F}$ is its flux, integration over a control volume gives

$$\frac{d}{dt} \int_{\Omega_i} q \, d\Omega + \int_{\partial\Omega_i} \boldsymbol{F} \cdot \boldsymbol{n} \, ds = 0, \tag{94}$$

where $\boldsymbol{n}$ is the outward unit normal to the control-volume boundary. This equation expresses the balance between the rate of change of the quantity inside a cell and its net flux through the cell boundary.

3.2 Governing Advection Equation

The horizontal advection of surface elevation is governed by

$$\frac{\partial h}{\partial t} + \boldsymbol{u}_H \cdot \nabla_H h = 0, \qquad t \in [t^k, t^k + \Delta t], \tag{95}$$

where h denotes the surface elevation during the advection step, $\boldsymbol{u}_H = (u, v)$ is the horizontal velocity vector, and $\nabla_H = (\partial/\partial x, \partial/\partial y)$ is the horizontal gradient operator.

During each advection interval, the velocity vector is assumed to be constant in space and time:

$$\boldsymbol{u}_H = (u, v) = \text{constant}. \tag{96}$$

Consequently,

$$\nabla_H \cdot \boldsymbol{u}_H = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0. \tag{97}$$

Using the product rule, the governing equation can therefore be written in conservative form:

$$\frac{\partial h}{\partial t} + \nabla_H \cdot (\boldsymbol{u}_H h) = 0. \tag{98}$$

The objective is to compute the surface elevation at $t^{k+1} = t^k + \Delta t$ from its known distribution at time t^k.

3.2.1 Integral Formulation

Let Ω_i be an arbitrary two-dimensional control volume. Integrating Equation (118) over Ω_i gives

$$\int_{\Omega_i} \frac{\partial h}{\partial t}\, d\Omega + \int_{\Omega_i} \nabla_H \cdot (\boldsymbol{u}_H h)\, d\Omega = 0. \tag{99}$$

Since the horizontal velocity vector $\boldsymbol{u}_H = (u, v)$ is constant in space, its divergence vanishes:

$$\nabla_H \cdot \boldsymbol{u}_H = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0.$$

Using the product rule,

$$\nabla_H \cdot (h\boldsymbol{u}_H) = \boldsymbol{u}_H \cdot \nabla_H h + h \nabla_H \cdot \boldsymbol{u}_H = \boldsymbol{u}_H \cdot \nabla_H h.$$

Therefore, the advection equation can be expressed in conservative form, and applying the divergence theorem to the second term yields

$$\frac{d}{dt} \int_{\Omega_i} h\, d\Omega + \int_{\partial\Omega_i} h\, \boldsymbol{u}_H \cdot \boldsymbol{n}\, ds = 0. \tag{100}$$

Assuming that the control volume is fixed in space, the first term can be expressed as

$$\int_{\Omega_i} \frac{\partial h}{\partial t}\, d\Omega = \frac{d}{dt} \int_{\Omega_i} h\, d\Omega. \tag{101}$$

Applying the divergence theorem to the second term yields

$$\frac{d}{dt} \int_{\Omega_i} h\, d\Omega + \int_{\partial\Omega_i} h\, \boldsymbol{u}_H \cdot \boldsymbol{n}\, ds = 0. \tag{102}$$

Define the cell-averaged surface elevation as

$$\overline{h}_i(t) = \frac{1}{|\Omega_i|} \int_{\Omega_i} h(\boldsymbol{x}, t)\, d\Omega, \tag{103}$$

where $|\Omega_i|$ is the area of control volume Ω_i. Since the control volume is fixed, its area is constant, and therefore

$$\frac{d}{dt} \int_{\Omega_i} h\, d\Omega = |\Omega_i| \frac{d\overline{h}_i}{dt}. \tag{104}$$

Substitution into Equation (119) gives the integral balance in terms of the cell average:

$$|\Omega_i| \frac{d\overline{h}_i}{dt} + \int_{\partial\Omega_i} h\, \boldsymbol{u}_H \cdot \boldsymbol{n}\, ds = 0. \tag{105}$$

3.2.2 Numerical Fluxes at Control-Volume Faces

Let the boundary $\partial\Omega_i$ consist of N_f faces, denoted by $f = 1, \ldots, N_f$. The boundary integral in Equation (105) can be decomposed into a sum of face integrals:

$$|\Omega_i| \frac{d\overline{h}_i}{dt} + \sum_{f=1}^{N_f} \int_f h\, \boldsymbol{u}_H \cdot \boldsymbol{n}_{if}\, ds = 0, \tag{106}$$

where $\boldsymbol{n}_{if}$ is the outward unit normal to face f with respect to control volume Ω_i.

The exact value of h along a face is generally unknown. Therefore, the face integral must be approximated using a numerical flux. Define the normal velocity at face f as

$$u_{n,if} = \boldsymbol{u}_H \cdot \boldsymbol{n}_{if}. \tag{107}$$

Because the velocity vector is constant during the advection interval, its normal component is determined by the orientation of the face.

A first-order upwind approximation is used to determine the surface elevation at each

face. For an interior face shared by cells i and j, the upwind value is

$$h_f^{\text{up}} = \begin{cases} \overline{h}_i, & u_{n,if} \geq 0, \\ \overline{h}_j, & u_{n,if} < 0. \end{cases} \tag{108}$$

Thus, the value is taken from cell i when the flow leaves cell i through face f, and from cell j when the flow enters cell i from its neighbor.

Approximating the face integral by the product of the face length, normal velocity, and upwind value gives

$$F_{if} = |f|\, u_{n,if} h_f^{\text{up}}, \tag{109}$$

where $|f|$ denotes the length of face f. The sign of $u_{n,if}$ accounts for the direction of transport relative to the outward normal of cell i.

For a boundary face, the upwind value is taken from the interior cell if the flow is outward. If the flow enters the domain, the value must be specified using an appropriate inflow boundary condition.

Substituting the numerical fluxes into Equation (106) gives the semi-discrete finite volume equation:

$$|\Omega_i| \frac{d\overline{h}_i}{dt} + \sum_{f=1}^{N_f} F_{if} = 0. \tag{110}$$

This equation is discrete in space but remains continuous in time. It states that the rate of change of the cell-averaged surface elevation is determined by the net advective flux through the cell boundary.

3.2.3 Explicit Euler Time Discretization

The temporal derivative in Equation (110) is discretized using the explicit Euler method:

$$\frac{d\overline{h}_i}{dt} \approx \frac{\overline{h}_i^{k+1} - \overline{h}_i^{k}}{\Delta t}, \tag{111}$$

where $\overline{h}_i^{k}$ and $\overline{h}_i^{k+1}$ denote the cell averages at times t^k and t^{k+1}, respectively.

Evaluating the numerical fluxes using the known solution at time t^k gives

$$|\Omega_i| \frac{\overline{h}_i^{k+1} - \overline{h}_i^{k}}{\Delta t} + \sum_{f=1}^{N_f} F_{if}^{k} = 0. \tag{112}$$

Rearranging this equation yields the explicit update:

$$\boxed{\overline{h}_i^{k+1} = \overline{h}_i^k - \frac{\Delta t}{|\Omega_i|} \sum_{f=1}^{N_f} F_{if}^k} \tag{113}$$

where

$$F_{if}^k = |f|\, u_{n,if} \left(h_f^{\text{up}}\right)^k. \tag{114}$$

Since the velocity is constant during the advection interval and the fluxes are evaluated using the solution at time t^k, all quantities on the right-hand side of Equation (113) are known. The update is therefore explicit.

3.2.4 Stability Considerations

The explicit Euler method combined with a first-order upwind flux requires a suitable Courant–Friedrichs–Lewy (CFL) condition. A common cellwise restriction is

$$\frac{\Delta t}{|\Omega_i|} \sum_{f=1}^{N_f} |f| \max(0, u_{n,if}) \leq C_{\text{CFL}}, \tag{115}$$

where C_{CFL} is a stability constant depending on the numerical scheme and mesh. For the standard first-order upwind method with forward Euler time integration, a typical choice is $C_{\text{CFL}} \leq 1$, subject to the mesh geometry and the assumptions of the discretization. The condition must hold for every control volume.

The time step must therefore be chosen according to the control-volume areas, face lengths, and normal components of the velocity. Although the velocity vector is constant during each advection interval, the allowable time step can vary with the mesh geometry.

In summary, the finite volume method transforms the horizontal advection equation into a system of evolution equations for the cell-averaged surface elevation. Spatial discretization is obtained by integrating the governing equation over each control volume and approximating the face fluxes using an upwind scheme. Explicit Euler time integration then advances the cell averages from t^k to t^{k+1}.

3.3 Finite Volume Advection Scheme Using Dual Control Volumes, Second-Order MUSCL–Hancock Reconstruction, the MC Slope Limiter, and Upwind Fluxes

3.3.1 Governing Equation

The horizontal advection of the surface elevation is described by

$$\frac{\partial h}{\partial t} + \boldsymbol{u}_H \cdot \nabla_H h = 0, \qquad t \in [t^k, t^k + \Delta t], \tag{116}$$

where h is the surface elevation, $\boldsymbol{u}_H = (u, v)$ is the horizontal velocity vector, and $\nabla_H = (\partial/\partial x, \partial/\partial y)$ is the horizontal gradient operator.

During each advection interval, the velocity is assumed to remain constant. Therefore,

$$\nabla_H \cdot \boldsymbol{u}_H = 0, \tag{117}$$

and the advection equation can be written in conservative form:

$$\frac{\partial h}{\partial t} + \nabla_H \cdot (\boldsymbol{u}_H h) = 0. \tag{118}$$

The objective is to calculate the surface elevation at $t^{k+1} = t^k + \Delta t$ from its known distribution at time t^k.

The numerical scheme is designed to achieve second-order spatial accuracy in smooth regions through MUSCL (Monotonic Upstream-centered Scheme for Conservation Laws) reconstruction with the MC (Monotonized Central) slope limiter. The Hancock predictor provides a time-centered approximation of the face states. The overall temporal accuracy, however, depends on the precise predictor and update procedure implemented in the code.

3.3.2 Median-Dual Control Volumes

The finite volume method (FVM) is based on integrating the conservation equation over control volumes. In the present approach, median-dual control volumes are constructed around the nodes of the computational mesh. Each node is associated with a control volume formed from portions of the surrounding mesh elements.

Let Ω_i denote the control volume associated with node i, and let $|\Omega_i|$ denote its area.

Integrating Equation (118) over Ω_i and applying the divergence theorem gives

$$\frac{d}{dt}\int_{\Omega_i} h\, d\Omega + \int_{\partial\Omega_i} h\boldsymbol{u}_H \cdot \boldsymbol{n}_i\, ds = 0, \tag{119}$$

where $\partial\Omega_i$ is the boundary of the control volume and $\boldsymbol{n}_i$ is its outward unit normal vector.

The first term represents the rate of change of the elevation within the control volume, whereas the second term represents the net advective flux through its boundary. Thus, the elevation changes according to the difference between the incoming and outgoing fluxes.

3.3.3 Fluxes Across Control-Volume Faces

The boundary of each control volume consists of a set of faces. Denoting a face by f, the integral balance can be written as

$$\frac{d}{dt}\int_{\Omega_i} h\, d\Omega + \sum_{f\subset\partial\Omega_i}\int_f h\boldsymbol{u}_H \cdot \boldsymbol{n}_{if}\, ds = 0, \tag{120}$$

where $\boldsymbol{n}_{if}$ is the outward unit normal to face f relative to control volume Ω_i.

The normal velocity at a face is defined as

$$u_{n,if} = \boldsymbol{u}_H \cdot \boldsymbol{n}_{if}. \tag{121}$$

Since the elevation at a face is not directly known, it must be estimated from the surrounding nodal values. In this implementation, a second-order MUSCL–Hancock scheme is used to reconstruct and predict the elevation at the faces before the numerical fluxes are evaluated.

3.3.4 MUSCL Reconstruction

The MUSCL (Monotonic Upstream-centered Scheme for Conservation Laws) approach improves spatial accuracy by reconstructing the variation of the elevation within each control volume. Instead of assuming a constant elevation throughout a control volume, a linear approximation is used:

$$h_i(\boldsymbol{x}) = h_i + \nabla h_i \cdot (\boldsymbol{x} - \boldsymbol{x}_i), \tag{122}$$

where h_i is the elevation associated with node i, $\boldsymbol{x}_i$ is its position, and ∇h_i is an approximation of the local elevation gradient.

This reconstruction provides an estimate of the elevation at a face using the values at nearby nodes. It captures local spatial variations more accurately than a piecewise-constant approximation, which is characteristic of a first-order finite volume scheme.

However, an unrestricted linear reconstruction may produce overshoots and undershoots near steep gradients. A slope limiter is therefore introduced to control the reconstructed gradient.

3.3.5 Monotonized Central (MC) Slope Limiter

The Monotonized Central (MC) limiter restricts the reconstructed slopes to reduce non-physical oscillations near sharp changes in the elevation field.

For a one-dimensional sequence of nodal values, the MC-limited slope can be expressed as

$$\sigma_i = \text{minmod}\left(2\Delta_{i-\frac{1}{2}}, \frac{\Delta_{i-\frac{1}{2}} + \Delta_{i+\frac{1}{2}}}{2}, 2\Delta_{i+\frac{1}{2}}\right), \tag{123}$$

where

$$\Delta_{i-\frac{1}{2}} = h_i - h_{i-1}, \qquad \Delta_{i+\frac{1}{2}} = h_{i+1} - h_i. \tag{124}$$

The minmod function returns the argument with the smallest magnitude when all arguments have the same sign; otherwise, it returns zero. Consequently, the reconstructed slope is reduced near local extrema and regions where neighboring gradients change sign.

In two dimensions, the corresponding limiter is applied to the gradient reconstruction according to the adopted discretization. The limited gradient is then used in Equation (122) to obtain the face values.

The MC limiter helps control spurious oscillations while retaining second-order accuracy in sufficiently smooth regions. It does not, by itself, guarantee that the numerical solution is free from all oscillations or errors.

3.3.6 Hancock Predictor

The Hancock step predicts the reconstructed elevation over half a time step. This prediction accounts for the evolution of the solution during the time interval and provides

time-centered values for evaluating the numerical fluxes.

For a linear reconstruction and a constant velocity, a representative half-step prediction at face f is

$$h_{i,f}^{k+\frac{1}{2}} = h_i^k + \nabla h_i^k \cdot (\boldsymbol{x}_f - \boldsymbol{x}_i) - \frac{\Delta t}{2} \boldsymbol{u}_H \cdot \nabla h_i^k, \tag{125}$$

where $\boldsymbol{x}_f$ denotes the face location and ∇h_i^k is the limited gradient at time t^k.

The first two terms reconstruct the elevation at the face, while the final term accounts for its predicted evolution over half a time step. This is the basic idea of the Hancock predictor; the precise predictor formula depends on the spatial reconstruction and its implementation.

Combining the linear reconstruction, the MC limiter, and the Hancock predictor produces the second-order MUSCL–Hancock approach used to calculate the face states.

3.3.7 Upwind Numerical Flux

Once the predicted face states are available, the numerical flux is calculated according to the direction of the normal velocity. For an interior face shared by control volumes i and j, the upwind state is selected from the side from which the flow originates:

$$h_f^{\mathrm{up}} = \begin{cases} h_{i,f}^{k+\frac{1}{2}}, & u_{n,if} \geq 0, \\ h_{j,f}^{k+\frac{1}{2}}, & u_{n,if} < 0. \end{cases} \tag{126}$$

If the normal velocity is positive, the flow leaves control volume i through the face, and the state reconstructed from side i is used. If the normal velocity is negative, the flow enters from the neighboring control volume j, and the state from that side is selected.

The numerical flux integrated over the face is approximated by

$$F_{if} = |f|\, u_{n,if}\, h_f^{\mathrm{up}}, \tag{127}$$

where $|f|$ is the length of the face. The sign of the normal velocity determines the direction of transport.

For an interior face shared by two control volumes, the same physical flux must be used with opposite signs in their respective balance equations. This ensures that the flux leaving one control volume enters its neighbor, preserving the discrete conservation property.

At physical boundaries, the face state is determined using the boundary treatment implemented in the numerical scheme. For inflow boundaries, the appropriate incoming elevation must be specified or consistently supplied by the adopted boundary treatment.

3.3.8 Finite Volume Update

Approximating the elevation integral by the control-volume area and its representative elevation, the semi-discrete balance becomes

$$|\Omega_i|\frac{dh_i}{dt} + \sum_{f\subset\partial\Omega_i} F_{if} = 0. \tag{128}$$

The numerical fluxes are evaluated using the reconstructed and predicted face states. The surface elevation is then updated according to the net flux through the boundary:

$$h_i^{k+1} = h_i^k - \frac{\Delta t}{|\Omega_i|}\sum_{f\subset\partial\Omega_i} F_{if}. \tag{129}$$

This relation expresses the central principle of the finite volume method: the elevation in each control volume changes according to the net advective flux through its faces. In the present method, the second-order spatial reconstruction and Hancock prediction are used in evaluating these fluxes.

3.3.9 Time-Step Restriction

As an explicit advection scheme, MUSCL–Hancock requires a suitable time step relative to the velocity and mesh resolution. This restriction is commonly expressed through a Courant–Friedrichs–Lewy (CFL) condition:

$$\frac{\Delta t}{|\Omega_i|}\sum_{f\subset\partial\Omega_i} |f|\,\max(0, u_{n,if}) \le C_{\mathrm{CFL}}, \tag{130}$$

where C_{CFL} is a scheme-dependent stability constant. Its appropriate value depends on the precise time integration, limiter, mesh geometry, and implementation.

The time step should therefore be selected according to the control-volume areas, face lengths, and normal velocity components. The condition must be checked for all control volumes.

3.3.10 Summary of the Numerical Procedure

The advection calculation proceeds as follows:

1. Construct the median-dual control volumes and determine their areas, face lengths, and normal vectors.
2. Estimate the local elevation gradients from neighboring nodal values.
3. Apply the MC limiter to control the reconstructed gradients near steep variations.
4. Use the MUSCL–Hancock procedure to reconstruct and predict the elevation at the control-volume faces.
5. Select the upwind state at each face and calculate the numerical advective flux.
6. Update the elevation in each control volume using the net flux through its boundary.

This procedure provides a conservative finite volume formulation for the horizontal advection of surface elevation, with second-order reconstruction and a limiter designed to reduce nonphysical oscillations.